Лабораторная работа №4
Базовые растровые алгоритмы

Веб-приложение для GitHub Pages · Vanilla JavaScript · VirtualGrid

Реализовано	Пошаговый, ЦДА, Брезенхем-линия, Брезенхем-окружность, Wu, Castle–Piteway (коника)
Архитектура	RasterEngine → VirtualGrid → GridPainter
Benchmark	100 000 отрезков + 50 000 окружностей, UI исключён
Проверка	21/21 автоматических тестов

Цель работы

Закрепить методы растеризации отрезков и кривых на дискретной сетке, реализовать вычисление пикселей без использования функций рисования примитивов, исследовать производительность и визуализировать влияние различных алгоритмов.

1. Архитектура проекта

RasterEngine содержит чистые статические методы. Единственный способ вывода результата — вызов IRasterTarget-подобного метода setPixel(x, y, intensity). Алгоритмы не получают ссылку на Canvas и не используют lineTo, arc и другие функции построения примитивов.

VirtualGrid хранит двумерную матрицу Float32Array[]. Значение 0 означает пустую ячейку, значения 0…1 — интенсивность. Координата (0,0) находится в центре массива.

GridPainter запускается только после алгоритма и отображает уже заполненную матрицу увеличенными квадратами, а также выводит сетку, оси и подписи.

Benchmark Worker запускает нагрузочные тесты в отдельном потоке. Время UI и построения графиков в измерение не входит.

Привязка координат

Целая координата (x, y) соответствует центру одной растровой ячейки. Границы этой ячейки проходят по x±0,5 и y±0,5. Экранный столбец вычисляется как x+originX, строка — originY−y.

2. Реализованные алгоритмы

2.1. Пошаговый алгоритм

Выбирается ведущая ось. Для пологой линии перебираются целые x и вычисляется y по уравнению прямой; для крутой — наоборот. Нецелая координата округляется до ближайшего центра пикселя.

2.2. ЦДА (DDA)

steps=max(|dx|,|dy|); xInc=dx/steps; yInc=dy/steps; x←x+xInc; y←y+yInc

Алгоритм работает во всех октантах, потому что знаки дробных приращений автоматически следуют знакам dx и dy.

2.3. Брезенхем для линии

Использована симметричная обобщённая форма: dx=|x1−x0|, dy=−|y1−y0|, sx и sy задают направления, err=dx+dy. На каждом шаге анализируется e2=2·err. Поэтому один код обрабатывает все 8 октантов, горизонтальные и вертикальные линии.

2.4. Брезенхем для окружности

Вычисляется один октант. Каждая точка (x,y) отражается в восемь симметричных положений относительно центра. Решающий параметр d обновляется целочисленными операциями.

2.5. Алгоритм Ву

Если идеальная линия пересекает очередной столбец в дробной координате y=n+f, два соседних пикселя получают интенсивности 1−f и f. В VirtualGrid хранится реальная интенсивность, поэтому GridPainter визуально показывает сглаженную границу.

2.6. Castle–Piteway / коническая кривая

Выполнена целочисленная инкрементная растеризация эллипса — конического сечения. Эллипс делится на две области по наклону; midpoint-параметр определяет выбор следующей клетки. Дробный член 1/4 устранён умножением решающей функции на 4, поэтому внутренний цикл остаётся целочисленным.

3. Сопроводительный пример вычислений

Рассмотрим линию от (-6,-3) до (7,5) алгоритмом Брезенхема.

dx=13; |dy|=8; sx=1; sy=1; в симметричной записи dy=-8; начальная ошибка err=dx+dy=5.

Шаг	x	y	err	e2	Действие
0	-6	-3	5	10	шаг x и y
1	-5	-2	10	20	шаг x
2	-4	-2	2	4	шаг x и y
3	-3	-1	7	14	шаг x
4	-2	-1	-1	-2	шаг x и y

На первом шаге e2=10. Поскольку $10 \geq -8$, выполняется шаг по x и err уменьшается на 8. Поскольку $10 \leq 13$, выполняется шаг по y и err увеличивается на 13. Получаем следующий пиксель (-5,-2). Полная таблица текущего примера автоматически строится в веб-интерфейсе.

4. Проверка корректности

Встроенный модуль самопроверки содержит 21 тест. ЦДА и Брезенхем проверяются в восьми октантах; дополнительно тестируются горизонтальная/вертикальная линии, восьмикратная симметрия окружности, четырёхкратная симметрия эллипса и наличие дробной интенсивности у алгоритма Ву. Контрольный запуск: 21/21 тестов.

5. Методика benchmark

Генератор использует детерминированный псевдослучайный набор. Для линий строится «веер» из начала координат: номер очередной линии циклически задаёт один из 8 октантов, а длина и малая компонента выбираются генератором. Полный тест достигает 100 000 отрезков. Для окружностей генерируются 50 000 центров и радиусов.

Для уменьшения влияния JIT перед измерением выполняется прогрев. Каждая точка графика измеряется три раза; используется медиана. Очистка растра выполняется до запуска таймера. Сам таймер охватывает только циклы RasterEngine + VirtualGrid.

6. Результаты контрольного запуска

Тест	Время, мс
Пошаговый, 100 000 линий	123.3
ЦДА, 100 000 линий	210.2
Брезенхем, 100 000 линий	141.8
Ву, 100 000 линий	469.4
Брезенхем, 50 000 окружностей	140.0

В этом запуске отношение времени ЦДА к Брезенхему для 100 000 линий составило 1.48. Значения являются контрольными и зависят от процессора, браузера/JIT и фоновой нагрузки; веб-приложение пересчитывает их на компьютере проверки.

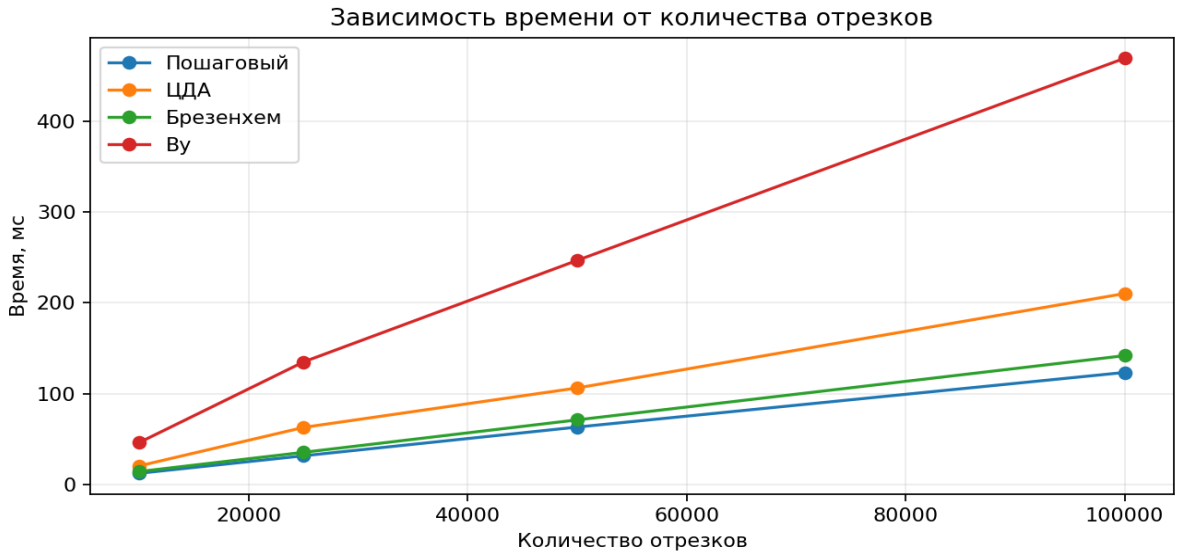


Рис. 1. Зависимость времени от количества отрезков.

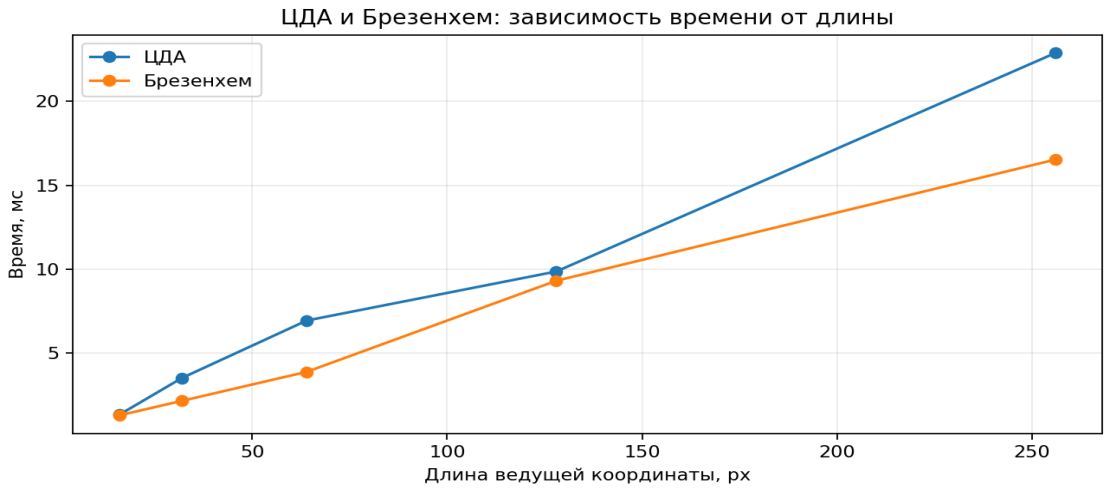


Рис. 2. Сравнение ЦДА и Брезенхема при росте длины.

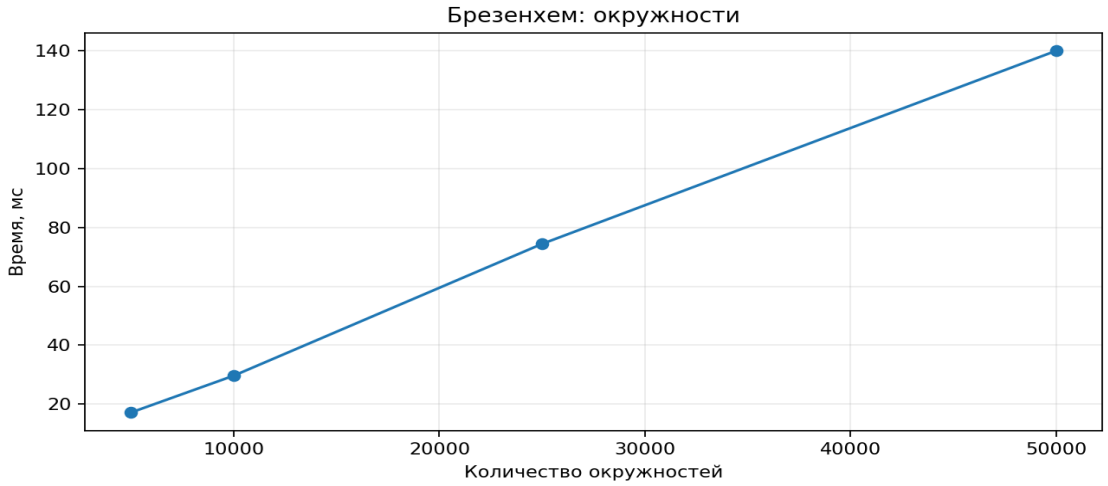


Рис. 3. Время построения окружностей.

7. Почему ЦДА медленнее Брезенхема

Оба алгоритма имеют асимптотику O(L), где L — число шагов по ведущей координате. Разница появляется в константных затратах внутреннего цикла.

ЦДА: хранит дробные x и y; выполняет два дробных приращения; перед SetPixel округляет обе координаты. Брезенхем: хранит целочисленную ошибку; использует сложение, вычитание, удвоение и сравнения; координаты всегда целые.

На современных JavaScript-движках Number формально является IEEE-754, поэтому точное соотношение зависит от JIT. Однако Math.round и поддержание дробных координат остаются дополнительной работой. Контрольный benchmark подтвердил ожидаемое преимущество Брезенхема: ЦДА был примерно в 1,48 раза медленнее на 100 000 отрезков, а график по длине сохраняет преимущество при увеличении L.

8. Вывод

В работе реализовано разделение вычисления растра и отображения, корректная обработка всех октантов, сглаживание Wu и целочисленная коника. Автоматический benchmark показывает масштабируемость времени с количеством примитивов и длиной линий. Архитектура соответствует схеме RasterEngine → VirtualGrid → GridPainter и подходит для публикации как статическое приложение GitHub Pages.

9. Источники

1. J. E. Bresenham. Algorithm for Computer Control of a Digital Plotter. IBM Systems Journal 4(1), 1965. DOI: 10.1147/SJ.41.0025.

2. Xiaolin Wu. An Efficient Antialiasing Technique. Computer Graphics 25(4), 1991. DOI: 10.1145/127719.122734.

3. M. L. V. Pitteway. Algorithm for Drawing Ellipses or Hyperbolae with a Digital Plotter. The Computer Journal 10(3), 1967. DOI: 10.1093/comjnl/10.3.282.